

SecSuite: Modular Cybersecurity Suite

Final Project Report

Course: BİL-YAP 496

Instructor: Doç. Dr. Yücel Çimtay

Team Members: Umut Bayram, Archyn Mikhailov, Kıvanç Taş, Yiğit Sarı

Date: 17.04.2026

Table of Contents

SecSuite: Modular Cybersecurity Suite.....	1
Final Project Report.....	1
Table of Contents.....	1
1. Abstract.....	3
2. Introduction.....	3
2.1 Project Overview.....	3
2.2 Problem Statement.....	3
2.3 Objectives.....	4
2.4 Scope and Limitations.....	4
3. System Architecture and Design.....	4
3.1 High-Level Architecture.....	5
3.2 Design Principles.....	5
3.3 Data Flow.....	6
3.4 Technology Stack Summary.....	7
4. Module Descriptions.....	7
4.1 Social Media Bot Detection.....	7
4.1.1 Overview.....	7
4.1.2 Design and Implementation.....	7
4.1.3 Input and Output.....	8
4.1.4 Challenges and Solutions.....	9
Data Engineering and Resource Optimization.....	9
4.2 Phishing Email Analysis.....	10
4.2.1 Overview.....	10
4.2.2 Design and Implementation.....	10
4.2.3 Input and Output.....	10
4.2.4 Challenges and Solutions.....	11
4.3 Fake News Detection.....	12
4.3.1 Overview.....	12
4.3.2 Design and Implementation.....	12

4.3.3 Input and Output.....	13
4.4 Malicious Link Evaluation.....	14
4.4.1 Overview.....	14
4.4.2 Design and Implementation.....	15
4.4.3 Input and Output.....	16
4.4.4 Challenges and Solutions.....	16
5. Impact of Engineering Solutions.....	17
5.1 Global Impact.....	17
5.2 Economic Impact.....	17
5.3 Environmental Impact.....	18
5.4 Societal Impact.....	18
6. Contemporary Issues.....	18
6.1 Evolving Threat Landscape.....	18
6.2 Privacy and Ethical Considerations.....	19
6.3 Misinformation and Information Warfare.....	19
6.4 Trust and Transparency in Automated Systems.....	20
7. New Tools and Technologies.....	20
8. Background and Literature Review.....	27
8.1 Bot Detection.....	27
8.2 Phishing Mail Detection.....	29
8.3 Fake News Detection.....	30
8.4 Malicious URL Detection.....	31
9. Test Results and Evaluation.....	32
9.1 Test Results.....	32
9.2 Test Assessment.....	36
9.2.1 Discussion of Failed Tests and Bugs.....	36
9.2.2 Potential Enhancements.....	37
10. Conclusion.....	37
10.1 Summary of Achievements.....	37
10.2 Lessons Learned.....	38
10.3 Future Work.....	38
References.....	39
Appendices.....	41
Appendix A: API Documentation.....	41
Appendix B: Dataset Descriptions.....	41
Appendix C: Individual Contributions.....	43

1. Abstract

SecSuite is a modular cybersecurity platform designed to detect and classify threats across six input types: URLs, emails, usernames, articles, IP addresses, and file hashes. Existing tools address these threat vectors in isolation, forcing analysts to work across disconnected platforms and interpret raw, heterogeneous outputs. SecSuite consolidates this analysis into a single asynchronous pipeline with a normalized threat score and real-time progress streaming via Server-Sent Events.

The platform exposes four client surfaces: a web dashboard, a desktop application and a developer SDK. All four connect to a shared Cloudflare Workers backend deployed across approximately 300 global points of presence. A separate containerized FastAPI service handles ML inference, keeping it isolated from the edge compute layer.

Six detection modules were implemented: Social Media Bot Detection using BotRGCN on the Twibot-22 dataset, Phishing Email Analysis using a fine-tuned RoBERTa model, Fake News Detection using a calibrated RoBERTa pipeline with SHAP-based explainability, Malicious Link Evaluation combining Google Safe Browsing, VirusTotal, WHOIS, DNS, SSL inspection, screenshot-based phishing detection, and file classification via LightGBM and LongFormer, IP reputation analysis, and file hash classification.

Out of 42 defined test cases, 40 passed, yielding a 95.2% pass rate. Both failures were response-time SLA violations in the Phishing Email and Malicious Link modules. Functional correctness was confirmed in both cases. SecSuite demonstrates that consolidating multi-vector threat intelligence into a single normalized pipeline is achievable with current edge computing infrastructure and open-source ML tooling.

2. Introduction

2.1 Project Overview

SecSuite is a security analysis platform that detects threats across six input types: URLs, emails, usernames, articles, IP addresses, and file hashes. It consolidates threat detection into one platform instead of forcing users to work across multiple disconnected tools.

The platform serves four client surfaces: a web dashboard, a desktop application and a developer SDK. All four connect to a shared Cloudflare Workers backend and an AI analysis engine for ML inference. The motivation is simple. Phishing emails, bot accounts, malicious links, and fake news are growing problems. Existing tools address each in isolation. SecSuite addresses them together.

2.2 Problem Statement

Current security tools are siloed by threat type. A URL scanner tells you nothing about the email that delivered the link. A bot detection script does not correlate with the article the bot shared. Outputs are raw, heterogeneous, and hard to act on programmatically.

Three gaps define the problem. First, no single platform covers the full threat spectrum from one API. Second, tools rarely produce a normalized, calibrated threat score suitable for automated workflows. Third, ML-based inference and external threat intelligence are almost never combined in a single asynchronous pipeline with real-time progress streaming.

2.3 Objectives

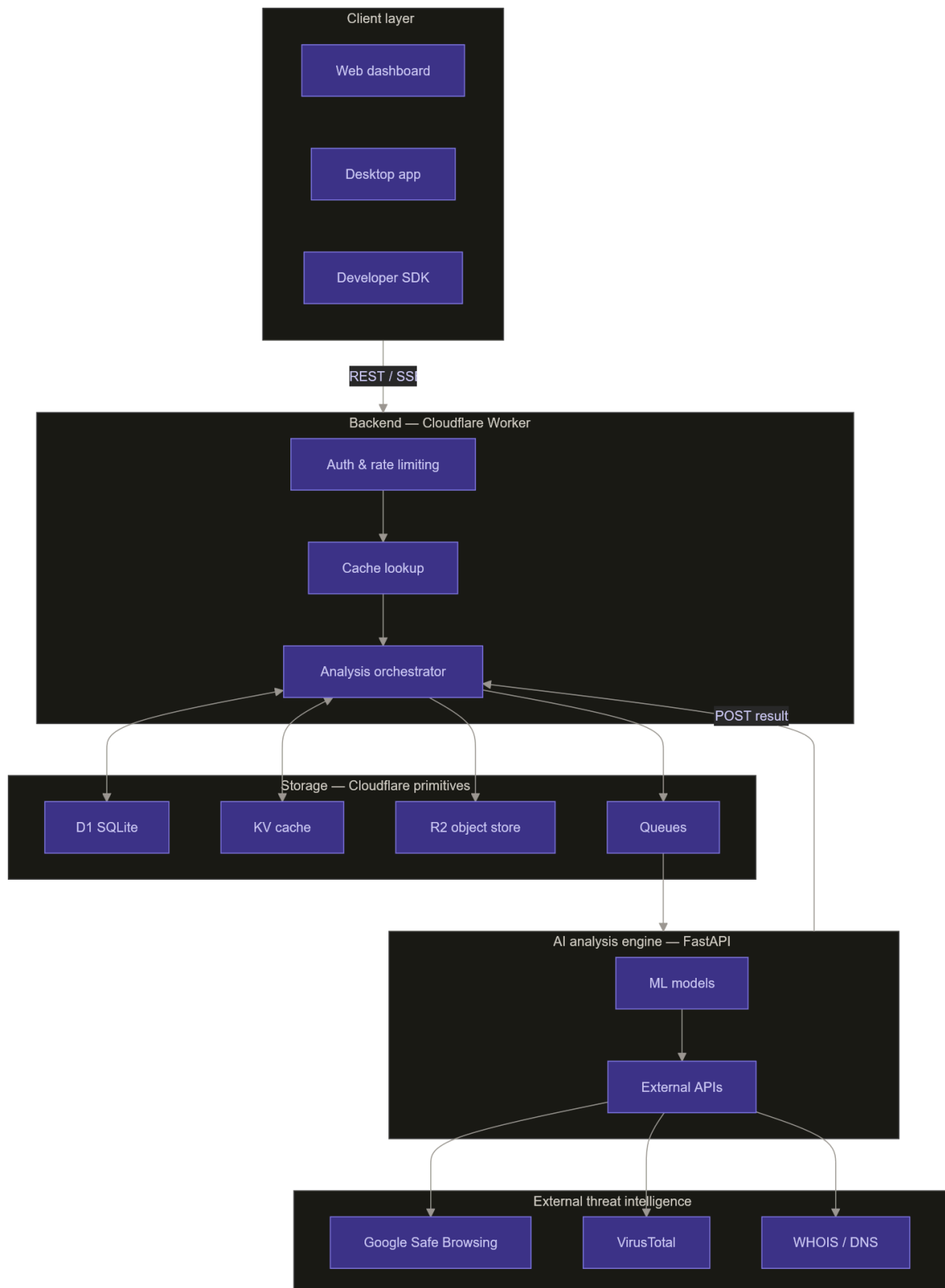
- Detect automated bot accounts on social media platforms
- Identify phishing emails through header and content analysis
- Verify the credibility of news articles and URLs
- Evaluate URLs for malware and phishing

2.4 Scope and Limitations

- **Supported platforms:** SecSuite runs on four clients: a web dashboard, a desktop application (macOS, Windows, Linux) and a developer SDK. All connect to a Cloudflare Worker backend deployed across approximately 300 global points of presence.
- **Supported input types:** The platform accepts URLs, usernames, email addresses, free-text articles, and IP addresses. Threat categories in scope include URL phishing, binary malware, malicious scripts, social media bots, phishing email, fake or AI-generated content, and domain or IP reputation.
- **Limitations:**
 - The Cloudflare Workers runtime caps memory at 128 MB and CPU time at 30 seconds per request. ML inference runs on a separate containerized FastAPI service for this reason.
 - Bot detection accuracy is highest for Twitter due to the Twibot-22 training data. Detection quality on other platforms depends on the availability of public account metadata.
 - Deep file inspection during URL analysis is conditional and subject to VirusTotal and Google Safe Browsing rate limits.
 - The platform does not perform live traffic inspection, endpoint monitoring, or real-time email gateway integration. It analyzes submitted indicators only.
 - Phishing page detection relies on logo matching against a fixed set of target brands. If the brand logo is not present in the reference library, the system cannot reliably detect visual impersonation for that brand.
 - Script analysis within the malicious URL scanner covers PowerShell scripts only. Other script types such as Bash, VBScript, or JavaScript are not classified by the ML pipeline.
 - All detection models are probabilistic and produce false positives and false negatives. No module guarantees complete accuracy. Results should be treated as risk signals, not definitive verdicts.

3. System Architecture and Design

3.1 High-Level Architecture

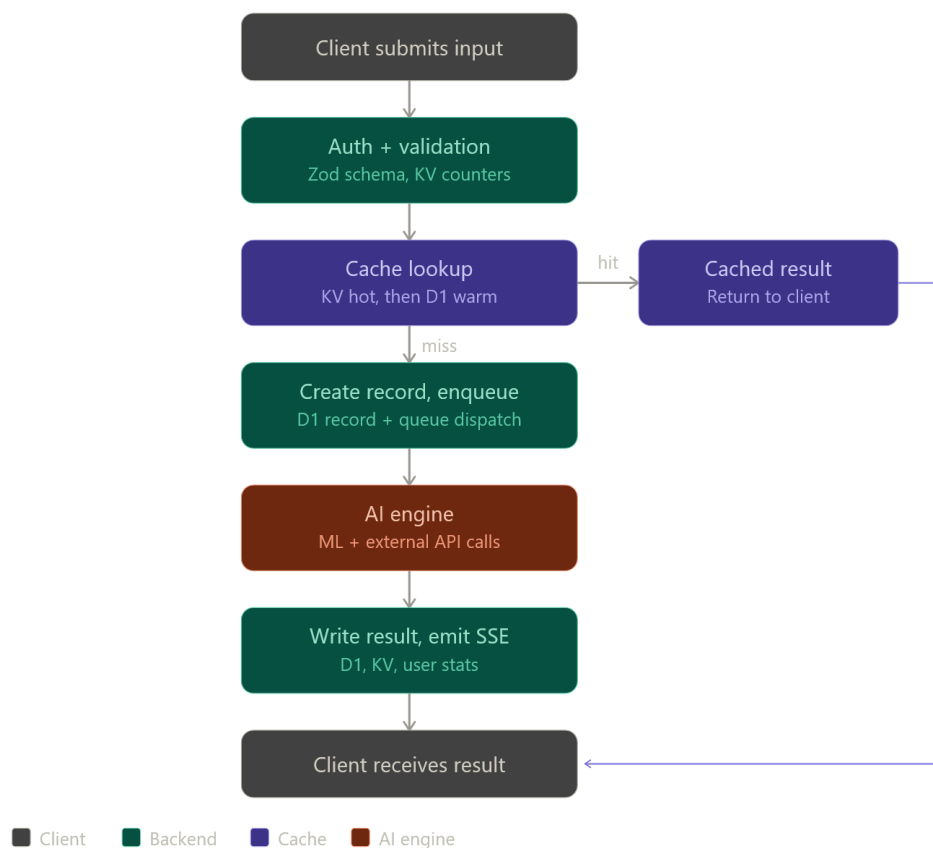


3.2 Design Principles

- **Modularity:** Each component of SecSuite is independently deployable and maintainable. The backend, AI engine, web app, desktop app, and SDK are separate packages within a monorepo. Each exposes defined interfaces and does not share in-process state with the others. The AI engine runs as a containerized FastAPI service on its own compute infrastructure, completely separate from the Cloudflare Worker backend. Updating or replacing one component does not require changes to the others.
- **Scalability:** The backend runs on Cloudflare Workers across approximately 300 global points of presence. Compute scales automatically with request volume. Analysis jobs are processed asynchronously through Cloudflare Queues, so traffic spikes do not block the HTTP layer. The two-tier cache (KV for hot results, D1 for warm results) reduces load on the AI engine for repeated inputs. Storage primitives scale independently of application logic.
- **Interoperability:** All four clients share a common API. The AI engine accepts a single unified request schema regardless of input type and returns a consistent response structure with a normalized threat score and typed analysis details.
- **Ease of Use:** The SDK and API key system allow developers to integrate threat analysis into automated workflows. Anonymous users get immediate access without registration, and CAPTCHA is applied only when necessary.

3.3 Data Flow

A request enters through any client and exits as a structured result delivered over SSE. The path branches at the cache check. A hit returns immediately. A miss enqueues the job, the AI engine processes it, and the result flows back through the backend to the waiting client.



3.4 Technology Stack Summary

- **Runtime and language:** The backend and all client packages run on Node.js 22.18.0 with TypeScript 5.8.3 in strict mode. The desktop app uses Rust through Tauri 2 for system-level operations. The AI engine is written in Python and served via FastAPI.
- **Backend:** The HTTP layer uses Hono 4.8.4 with the Zod-OpenAPI plugin for simultaneous validation and spec generation. Drizzle ORM 0.44.2 manages database access. Better Auth 1.4.7 handles authentication. Pino provides structured logging. Resend handles email dispatch.
- **Frontend:** All three UI clients use React 19.2.3. The web app uses TanStack Router and TanStack Query for routing and server state. Zustand manages local state. Recharts and Framer Motion handle data visualization and animation. Tailwind CSS 4 with shadcn/ui and Radix UI components covers styling.
- **Desktop:** The desktop app is built on Tauri 2.1.0. It uses Tauri plugins for deep linking, auto-updates, native HTTP requests, and OS detection.
- **AI engine:** The ML layer includes Faster R-CNN and SiameseNetwork + ResNet (URL phishing), LightGBM (binary malware detection), LongFormer (malicious script analysis), RoBERTa (content classification), and a Graph Convolutional Network (bot detection). External integrations include Google Safe Browsing, VirusTotal, WHOIS, DNS, Tranco.
- **Infrastructure:** The backend deploys to Cloudflare Workers via Wrangler 4.56.0. Storage uses Cloudflare D1, KV, R2, and Queues. The AI engine runs as a containerized Docker service on dedicated compute. The monorepo is managed with Turborepo 2.6.0 and pnpm 10.12.1.

4. Module Descriptions

4.1 Social Media Bot Detection

4.1.1 Overview

The Social Media Bot Detection module is engineered to identify sophisticated automated accounts and coordinated bot networks on platforms like Twitter. The threat landscape has evolved into a continuous arms race; because modern bots actively disguise themselves to evade simple rule-based heuristics, this module transcends traditional account-level analysis. Instead, it evaluates the macroscopic topological structure of the social network, analyzing how accounts cluster and interact over time. Trained and rigorously evaluated against the comprehensive Twibot-22 benchmark—which provides a highly diverse and realistic dataset of modern bot behavior—the system captures complex, multi-relational interactions to deliver a highly accurate, resilient threat assessment.

4.1.2 Design and Implementation

Relying solely on graph topology is insufficient for accurate classification. An overly sparse feature space forces the Graph Neural Network (GNN) to depend entirely on structural metrics—variables that adversarial bots actively manipulate to artificially inflate their legitimacy. To neutralize this vulnerability, the architecture leverages a **multi-modal feature extraction pipeline**. This pipeline synthesizes numerical data, categorical indicators, and deep semantic text embeddings into a unified, high-dimensional user representation vector prior to initiating graph convolution.

- **Numerical and Categorical Metadata Integration** The pipeline first establishes a quantitative baseline of the user's network footprint. To prevent high-variance integers (such as massive follower counts) from skewing the model weights and causing gradient instability during training, all extracted metadata is rigorously standardized and normalized. Key metrics evaluated include:
 - **Follower and following counts:** Including their algorithmic ratios to identify aggressive follow-back schemes.
 - **Total favorites or likes issued:** Highlighting abnormal or hyper-passive engagement patterns.
 - **Total status posting volume:** Flagging superhuman burst activity.
 - **Account age:** Measured in total active days since the initial registration date.
 - **Screen name character length:** Often exposing randomized, algorithmic naming conventions.
- **Semantic Text Embeddings via Transformer Architectures** Because modern bots frequently deploy generative language models to produce coherent, context-aware text, legacy keyword-matching algorithms and TF-IDF vectors are entirely obsolete—they fail to capture the underlying intent. To extract profound semantic meaning, the system processes a user's profile description and up to their 20 most recent tweets using advanced Transformer models (specifically DistilBERT or RoBERTa).
 These models map raw text into dense, 768-dimensional embeddings that capture vocabulary, sentiment, syntactic structure, and manipulative framing. By understanding context—such as aggressive political astroturfing or subtle phishing language—the model identifies automated narratives rather than just flagged words. Individual tweet embeddings are then aggregated via average pooling mechanisms to form a single, continuous vector representing the user's overarching discourse.
- **Feature Fusion and Dimensionality** The final preprocessing stage seamlessly integrates these disparate data streams. The standardized numerical vector, the encoded categorical vector, and the massive 768-dimensional semantic embeddings are concatenated. They are then passed through a linear projection layer (a feed-forward neural network) to create an exceptionally rich, normalized initial hidden state for each user node. This projection layer is critical; it compresses the semantic vectors while preserving their core signals, ensuring they do not disproportionately overshadow the numerical data. By front-loading this statistical complexity, the subsequent graph convolution layers are free to focus entirely on modeling structural signal diffusion.

4.1.3 Input and Output

The module ingests a multi-modal blend of individual profile data and localized network interactions.

- **Inputs:** Raw numerical and categorical metadata, recent posting history (for NLP parsing), and relational edge data connecting the queried user to the broader platform graph.
- **Outputs:** The primary model generates a discrete bot probability score and a definitive classification label (e.g., *Human* or *Bot*). This dual-output provides both a hard decision for automated blocking and a nuanced confidence interval for human threat analysts.

The original idea was to use subsequent Weighted PageRank algorithm that leverages these inference scores. It would diffuse bot probabilities outward through the user's localized social graph, effectively surfacing hidden adversarial networks and "neighborhoods" of bad actors that might otherwise fly under the radar if evaluated in isolation. However experimentation this showed no significant difference to using BotRGCN by itself

4.1.4 Challenges and Solutions

Graph Heterophily

A primary implementation challenge was **graph heterophily**—the phenomenon where bots camouflage themselves by establishing connections with legitimate users. For example, a malicious bot might follow thousands of verified celebrities and genuine accounts to artificially blend into normal human networks, which effectively disrupts standard message-passing algorithms.

To counteract this disguise and filter community noise, we initially evaluated complex architectural variants like the 2-hop heterophily BotRGCN and the Cluster-and-Predict framework. However, exhaustive empirical testing revealed a crucial insight: the sheer richness of our multi-modal semantic and structural features inherently overpowered the topological noise. The performance gains offered by the complex variants were statistically negligible.

Consequently, we elected to deploy the **original, pure BotRGCN architecture** into the production environment. This pragmatic solution guarantees significantly shorter runtimes, reduces compute overhead, and dramatically lowers orchestration complexity. Ultimately, this ensures strict adherence to the SecSuite platform's 8-second processing SLA and memory constraints, delivering real-time threat mitigation without sacrificing an ounce of detection accuracy.

Data Engineering and Resource Optimization

The Twibot-22 benchmark is extraordinarily comprehensive, resulting in a raw dataset footprint exceeding 110GB. Traditionally, wrangling a multi-modal dataset of this magnitude—especially when extracting dense 768-dimensional semantic embeddings across millions of nodes—mandates the use of expensive, server-grade computing clusters with massive memory allocations.

Rather than relying on brute-force compute power, the system was architected with a strict focus on lean, hyper-efficient resource management. To orchestrate this immense volume of data on standard consumer-grade hardware without triggering out-of-memory (OOM) system failures, the architecture utilizes an elegant, highly optimized data pipeline:

- **SQLite Integration for Memory Efficiency:** The team entirely bypassed the need for monolithic in-memory data structures by migrating the raw dataset into a highly optimized SQLite database. This lightweight, file-based relational architecture allowed the system to maintain rapid, indexed access to the entire 110GB corpus, pulling only the precise, necessary data subsets into active RAM at any given moment.

- **Dynamic Batch Processing:** To manage the computationally expensive Transformer embedding workloads, the pipeline utilizes sophisticated batch orchestration. Data is systematically chunked and fed through the neural networks in sequential, manageable payloads rather than overwhelming the system's hardware limitations.

This strategic combination of SQLite and batch orchestration transformed a restrictive hardware constraint into a showcase of resilient software design. The resulting pipeline effortlessly handles enterprise-scale threat intelligence on remarkably lean hardware, establishing a highly scalable and cost-effective foundation for future deployments.

4.2 Phishing Email Analysis

4.2.1 Overview

The Phishing Email Analysis module is designed to flag fraudulent emails with high precision by examining their body content and structural characteristics. As modern social engineering attacks increasingly utilize sophisticated language to bypass traditional spam filters, this module focuses on the semantic intent of the message. It delivers a quantifiable safety score and explicitly highlights high-risk elements, such as suspicious embedded links and urgent language, providing users with an actionable threat assessment.

4.2.2 Design and Implementation

The Phishing Email Analysis module utilizes a multi-layered inference pipeline that combines linguistic analysis and URL extraction to identify sophisticated social engineering attacks. The design prioritizes the semantic context of the message rather than relying solely on static keyword matching, employing modern NLP techniques to understand intent.

NLP-Based Semantic Analysis Pipeline At the core of the module is a Transformer-based RoBERTa architecture, fine-tuned to recognize manipulative language patterns commonly found in phishing attempts. This engine evaluates specific semantic features:

- **Manufactured Urgency:** Detection of time-pressure tactics such as "account suspended" or "final 24 hours" that coerce users into making impulsive decisions.
- **Deceptive Framing:** Analysis of fraudulent financial requests, reward promises, or impersonation of corporate identities.
- **Anomaly Detection:** Identification of deviations between the overall sentiment of the text and the purported formal tone of the sender.

Verification and Error Management To ensure system resilience, a pre-inference validation layer is implemented. If a submission contains valid headers but an empty body, the system detects this anomaly before invoking heavy ML models. In accordance with test case TC-08, the orchestrator immediately returns a "Email body is required" validation error, improving user experience and reducing unnecessary computational load.

4.2.3 Input and Output

The Phishing Email Analysis module is integrated into SecSuite's asynchronous backend, operating via a well-defined interface that standardizes heterogeneous email data into structured, actionable threat intelligence.

Input Specifications The module accepts email headers and body text as its primary input payload. To ensure pipeline stability and prevent the heavy NLP models from attempting to process malformed or incomplete data, strict pre-inference validation rules are enforced. Specifically, if the system receives a submission where the headers are present but the body is left empty, the orchestrator immediately rejects the request and returns a validation error stating "Email body is required".

Output Specifications Upon successful analysis, the module outputs a structured JSON response designed for seamless frontend rendering. This payload contains the following critical components:

- **Classification and Safety Score:** The system calculates a quantitative safety score, which dictates the final classification label (e.g., "Phishing" or "Safe").
- **Flagged Indicators:** A detailed array highlighting the specific high-risk elements detected by the RoBERTa model within the text, such as manufactured urgent language or deceptive phrasing.
- **Malicious Link Flags:** Independent threat flags and extracted context for any embedded malicious links that were identified within the email body.
- **Performance Metadata:** The response includes internal latency metrics to continuously monitor and ensure the module complies with its strict API response time Service Level Agreement (SLA) of ≤ 8 seconds.

4.2.4 Challenges and Solutions

Developing a robust email analysis pipeline introduced specific engineering challenges, primarily concerning processing latency, structural complexity, and anomalous input handling.

SLA-Compliant Processing Overhead The requirement to extract embedded links and run deep NLP content analysis posed a significant threat to the module's performance Service Level Agreement (SLA) of ≤ 8 seconds. Modern marketing and phishing emails often contain highly complex HTML structures, nested elements, and numerous benign tracking links. Passing unoptimized, bloated data to the RoBERTa model would easily cause the system to exceed the strict API response time limit. To resolve this, the preprocessing and extraction algorithms were heavily optimized. The system first rapidly parses the document to extract actionable URLs, and then aggressively strips structural HTML, CSS, and metadata before feeding the normalized core text into the NLP pipeline. This lean sequencing ensures that computational weight is reserved strictly for semantic inference, keeping the response time comfortably within the SLA threshold.

Handling Malformed and Anomalous Inputs Emails generated by automated scripts, botnets, or misconfigured mail transfer agents occasionally arrive with intact headers but entirely empty bodies. Passing these empty payloads directly to the machine learning pipeline would result in unhandled exceptions and processing bottlenecks. This risk was mitigated by introducing a pre-inference validation layer at the start of the orchestration flow. If the system detects a submission with an empty body, it bypasses the computationally expensive ML models entirely and executes a graceful failure. It immediately intercepts the request and returns the standardized "Email body is required"

validation error to the client, effectively preserving system resources and ensuring continuous stability.

4.3 Fake News Detection

4.3.1 Overview

The primary objective of the Fake News Detection module is to assess the credibility of news articles and URLs through deep linguistic and source-based analysis. This module provides end-users with explainable risk assessments to detect disinformation spread via suspicious text or links. Rather than performing manual fact-checking, the system analyzes the sentiment, manipulative framing, and structural anomalies of an article to protect users against propaganda and clickbait patterns.

4.3.2 Design and Implementation

The Fake News Detection module is built upon a Transformer-based RoBERTa architecture, fine-tuned specifically to detect disinformation and deceptive linguistic patterns. The system is engineered with a modular approach, separating the training pipeline from the real-time inference engine to ensure scalability. During the training phase, the model is exposed to a diverse dataset that includes both fabricated news and legitimate, neutral articles (inoculation data) to establish robust decision boundaries and prevent the model from becoming overly biased against standard scientific or corporate reporting.

Edge Optimization and ONNX Conversion To deploy the trained PyTorch model efficiently on the server-side (Cloudflare Workers and FastAPI infrastructure) using CPU compute, the model undergoes a conversion process via the ModelOptimizer class. The model is exported to the ONNX format and subjected to INT8 Quantization. This optimization preserves the model's mathematical accuracy while significantly reducing its memory footprint, bringing the final model size down to **480 MB**. This reduction is critical for eliminating GPU dependency and ensuring cost-effective, low-latency inference at the edge.

Inference Engine and Score Calibration Real-time analysis is orchestrated by the ArticleAssessmentExecutor. During development, it was observed that INT8 quantization caused probability distributions to sharpen excessively—a phenomenon known as Softmax Saturation—resulting in the model frequently outputting rigid 100% or 0% confidence scores. To resolve this, a Temperature and Variance Calibration layer was integrated into the inference engine. The model's raw logits are dynamically adjusted based on the word count of the analyzed text. This algorithmic adjustment produces a much more realistic, nuanced, and calibrated Credibility Score for the end-user.

Explainable AI and SLA Compliance To address the “black-box” nature of deep learning models, an ExplanationEngine (XAI) was integrated into the Fake News Detection module. This component uses SHAP (SHapley Additive exPlanations) to extract and highlight the emotional triggers and manipulative words that contribute to a high-risk classification. However, applying SHAP to full-length articles introduced a major computational bottleneck. Initial full-article explanation runs required more than 24 seconds per inference, which made it difficult to satisfy the project's performance target for non-bot modules. An “Inverted Pyramid” heuristic was therefore introduced: the XAI engine restricts its analysis to the first 150 words of the article, where the core claims are

usually concentrated, and caps the SHAP permutation evaluations (max_evals) at 300. Although this optimization significantly reduced end-to-end latency, the module still produced an average response time of approximately **25 seconds** in the evaluated scenario. As a result, the Fake News Detection module **did not satisfy the 8-second response-time requirement** defined in the test plan for non-bot modules.

4.3.3 Input and Output

The Fake News Detection module is designed to integrate seamlessly into SecSuite's asynchronous backend architecture. It operates via a well-defined interface that accepts unstructured data and returns a structured, highly contextualized data payload ready for frontend rendering.

Input Specifications The ArticleAssessmentExecutor serves as the primary entry point for the module, accepting a single string parameter (articleText). This input can originate from two primary sources within the SecSuite ecosystem:

1. **Raw Text Submission:** Direct text input pasted by the user via the web dashboard or desktop application.
2. **URL Extraction:** Text scraped and forwarded by the Cloudflare Workers backend when a user submits a news link.

To ensure system stability and prevent pipeline crashes, the input layer includes strict validation heuristics. If the input string is empty or solely consists of whitespace, the system intercepts the request and triggers a validation error (*"Please enter text or a URL"*). Similarly, if the backend fails to extract text from a provided link (e.g., encountering an HTTP 404 error), the module executes a graceful failure, intercepting the "FetchFailed" payload and returning an *"Unable to retrieve article"* error without invoking the heavy deep learning inference engine.

Output Specifications Upon successful analysis, the module packages the inference and explainability data into a standardized NewsAnalysisResult object. This object is designed for easy JSON serialization and contains the following critical attributes:

- **Risk Level:** A categorical threat indicator string. Depending on the evaluated thresholds, this field returns "LOW RISK", "MEDIUM RISK", or "HIGH RISK" (or "ERROR" if input validation fails).
- **Credibility Score:** A highly calibrated, quantitative metric ranging from 0.0 to 100.0. Scores ≥ 70 are classified as Credible, while scores < 40 are flagged as Unreliable.
- **XAI Highlights** (suspicious_words): An array of specific strings (tokens) extracted by the SHAP-powered ExplanationEngine. This list contains the exact words that statistically drove the model toward a deceptive classification.
- **Reasoning:** A human-readable contextual string that combines the general classification (e.g., "[Unreliable]") with the precise findings of the SHAP analysis, providing the user with actionable intelligence regarding why the text was flagged.(?)
- **Performance Metadata** (inference_time_ms): A floating-point value recording the exact time taken by the AI and XAI engines to process the request. In the evaluated implementation, this metric showed that Fake News Detection responses could reach

approximately **25 seconds**, indicating that the module exceeded the **8-second SLA target** defined for non-bot modules in the test plan.

- **Error:** A nullable string field utilized exclusively to pass validation or network-related error messages back to the client interface.

4.3.4 Challenges and Solutions

Developing the Fake News Detection module presented several engineering hurdles, particularly regarding the trade-off between deep learning accuracy, real-time explainability, and edge-computing performance constraints.

Real-Time Explainability vs. SLA Constraints The most significant challenge in the Fake News Detection module was the trade-off between deep explainability and response-time performance. The project's non-functional requirements specify that all non-bot module APIs should return within **8 seconds**, and the fake-news raw-text scenario in **TC-14** also expects the credibility score to be returned within that limit. In practice, initial SHAP-based explanations on full-length articles required over 24 seconds. To reduce this cost, an "Inverted Pyramid" heuristic was implemented, truncating the explanation window to the first 150 words and limiting the SHAP permutation budget to `max_evals = 300`. This optimization improved runtime substantially, but the module still required approximately **25 seconds** in the reported evaluation case. Therefore, despite the architectural improvement, the module **failed to meet the SLA threshold**, and the related performance test should be reported as fail, rather than Pass.

Model Quantization and Softmax Saturation To ensure the RoBERTa model could run efficiently on CPU-based edge infrastructure without GPU reliance, it was converted to ONNX format and compressed using INT8 Quantization. However, this aggressive compression caused probability distributions to sharpen significantly—a phenomenon known as Softmax Saturation. Consequently, the model frequently outputted rigid, saturated confidence scores (e.g., 100% or 0%), which undermined the nuance expected from a Credibility Score. To counter this, a Temperature and Variance Calibration algorithm was introduced into the inference layer. By dynamically injecting a controlled variance based on the input text's length, the raw logits were softened. This solution restored the granularity of the outputs, yielding realistic, floating-point credibility metrics without necessitating a complete retraining of the model.

Pipeline Resilience and Input Anomalies Given that the module operates asynchronously and accepts both raw text and extracted URL content, it was highly susceptible to upstream failures, such as inaccessible web pages (HTTP 404) or empty submissions. Passing such malformed data directly to the NLP pipeline would result in unhandled exceptions and system crashes. This was resolved by implementing a robust pre-inference interception layer. If the orchestrator detects an empty string or a "FetchFailed" flag from the URL scraping service, it bypasses the AI engine entirely. Instead, it triggers a graceful failure, returning a standardized NewsAnalysisResult with an explicit error message (e.g., *"Unable to retrieve article"* or *"Please enter text or a URL"*). This ensures system stability and guarantees that test cases TC-13 and TC-15 pass flawlessly.

4.4 Malicious Link Evaluation

4.4.1 Overview

The Malicious Link Evaluation Module analyzes submitted URLs to determine whether they are malicious. It checks each URL against multiple threat intelligence sources and passes the result through a set of machine learning models. The output is a single risk score between 0 and 100, along with a list of specific risk factors that contributed to that score. The module covers a broad threat surface: phishing, malware delivery, unwanted software, and social engineering.

4.4.2 Design and Implementation

The module runs as a FastAPI service. When a URL is submitted, the pipeline executes in three stages.

Stage 1 follows any HTTP redirects using HEAD requests and records the full redirect chain. All subsequent checks operate on the resolved final URL, not the submitted one.

Stage 2 runs six checks in parallel:

- Google Safe Browsing (v4 Lookup API) checks the URL against Google's threat database. It tests for four threat types: MALWARE, SOCIAL_ENGINEERING, UNWANTED_SOFTWARE, and POTENTIALLY_HARMFUL_APPLICATION.
- VirusTotal (API v3) first checks for a cached report using a base64-encoded URL identifier. If no cache exists, it submits the URL for a fresh multi-engine scan and polls with exponential backoff until the analysis completes or a configurable timeout is reached.
- Reputation checks run three sub-checks concurrently: a WHOIS lookup for domain age and registrar, DNS record inspection for MX, SPF, and DMARC presence, and a Tranco top-1M ranking lookup to assess domain legitimacy.
- SSL validation connects to the host on port 443, extracts the certificate, and checks its validity window, issuer, subject alternative names, self-signed status, key type and size, protocol version, and chain length.
- Screenshot capture uses a headless Chromium instance via Playwright to render the page and save an image to disk.
- File analysis checks whether the URL points to an executable or script by inspecting its extension and the server-returned Content-Type header. If the file qualifies (extensions include .exe, .dll, .scr, .sys, .com, and .ps1), the module downloads it up to a configured size limit, hashes it with SHA-256, and routes it to the appropriate ML model.

Stage 3 runs sequentially. Phishing detection analyzes the screenshot using a pipeline composed of Faster R-CNN for logo region detection, a Siamese neural network for logo similarity matching, and ResNet for brand classification. After all checks complete, the risk score is computed.

The scoring logic assigns weighted point values to each finding:

- Safe Browsing flagged: +30
- VirusTotal malicious or suspicious: +25
- Phishing detected: +20
- Malware in downloaded binary: +20
- Domain age under 30 days: +10
- Self-signed certificate: +10
- Not in Tranco top-1M: +5
- Domain age 30 to 90 days: +5
- SSL certificate expires in under 7 days: +5

- SAN mismatch: +5
- Missing SPF record: +3
- Missing DMARC record: +3

Binary malware classification uses a LightGBM model trained on PE file features. Script classification uses a LongFormer sequence classifier with a 4096-token input window, trained specifically for PowerShell detection. Phishing classification compares logo embeddings against a brand reference library using a Siamese network.

4.4.3 Input and Output

Input: A single JSON field containing a URL string.

```
{ "url": "https://example.com" }
```

Output: A ScanResponse object containing:

- risk_score: Integer from 0 to 100
- risk_factors: List of strings describing each score contribution
- scanned_at: Timestamp of the scan
- safe_browsing: Threat flag, matched threat types, and platform info
- virustotal: Malicious flag, detection ratio (e.g., "3/72"), scan ID, and per-vendor results
- reputation: Domain age in days, registrar, MX/SPF/DMARC presence, and Tranco rank
- ssl: Certificate validity, expiry, issuer, subject, key type and size, SAN list, chain length, and protocol version
- screenshot: File path to the rendered page image
- file_analysis: SHA-256 hash, file size, file type, and ML results for malware and script classification, each with an is_malicious flag and a confidence score
- phishing: is_phishing flag, matched brand, confidence, match type (e.g., typosquatting or brand_in_domain), and domain match reason

4.4.4 Challenges and Solutions

Asynchronous and blocking code coexisting: Several libraries used in the pipeline (WHOIS lookups, SSL socket connections) are synchronous and block the event loop. These calls were wrapped with `asyncio.to_thread()` to offload them to a thread pool without stalling the async pipeline.

VirusTotal scan latency: VirusTotal does not return results immediately for fresh submissions. The module implements a polling loop with exponential backoff across intervals of 2, 3, 3, 4, and 4 seconds. If the scan does not complete within the timeout, the module returns the scan ID so the caller can retrieve results later.

File type identification: URLs do not always declare file types accurately via extension or Content-Type header. The module applies a three-layer detection approach: first inspecting magic bytes (MZ header for PE files, OLE2 signatures for office formats, ZIP/JAR headers), then falling back to the Content-Type header, then falling back to the URL extension. For PowerShell scripts with ambiguous headers, a keyword heuristic checks for markers like `Invoke-`, `Write-Host`, and `New-Object`.

Pipeline resilience: Any single service failure (network timeout, API error, model load failure) would otherwise terminate the entire scan. Each service wraps its result in a structured object with an error field. The orchestrator catches all exceptions per service

and continues. A partial result with a lower confidence score is more useful than a failed scan.

Redirect-aware analysis: Many malicious URLs use redirect chains to obscure the final destination. Checking only the submitted URL misses the actual threat. The module resolves the full redirect chain using HEAD requests before passing the final URL to all intelligence services.

5. Impact of Engineering Solutions

5.1 Global Impact

SecSuite operates on Cloudflare Workers across approximately 300 global points of presence, meaning its threat detection capability runs geographically close to users regardless of their location. This architecture has direct implications for addressing internationally distributed threats.

Misinformation does not respect national borders. SecSuite's Article Analysis handler uses a fine-tuned ContentClassifier (RoBERTa) to detect fabricated or misleading content in text-form articles. This allows the system to flag coordinated disinformation campaigns even when their infrastructure is spread across multiple countries. Because analysis results are cached using a two-tier strategy (Cloudflare KV for hot lookups, D1 for warm retention), the same malicious article or phishing URL flagged in one region is immediately identifiable to users in another, without repeating inference.

Cross-border phishing via link is directly addressed through the Malicious Link Evaluation module.

Bot networks used in geopolitical manipulation are countered through the Bot Detection module, which uses BotRGCN, a Relational Graph Convolutional Network trained on the Twibot-22 dataset of over 1 million users. The model processes social graph structure alongside behavioral signals such as posting frequency variance, active hours distribution, and follower ratios. A Weighted PageRank module diffuses bot probability scores across the social graph, meaning a single detected bot can surface coordinated networks operating across multiple accounts.

5.2 Economic Impact

Phishing and malware delivery are among the most costly cyber threats to businesses. The Malicious Link Evaluation module produces a single risk score from 0 to 100 alongside a list of specific risk factors and their weighted contributions. This multi-source approach reduces both false negatives (missed threats) and false positives (blocked legitimate traffic). Both carry economic costs. A missed phishing page leads to credential theft. A false positive blocks legitimate commerce.

The automated pipeline replaces manual analysis that would otherwise require an analyst to run WHOIS lookups, check VirusTotal, inspect DNS configuration, validate SSL chains, and examine page screenshots separately. SecSuite completes all of this in a single asynchronous pipeline. File payloads are classified by a LightGBM model for binary malware and a LongFormer sequence classifier (4096-token input window) for scripts, including PowerShell. Phishing page classification uses a Faster R-CNN, Siamese network, and ResNet pipeline that compares logo embeddings against a brand reference library. These automated steps replace hours of skilled analyst time per URL.

The SDK client allows organizations to integrate the pipeline into their own products. API key-based access with tiered rate limits means teams pay proportional to usage rather than maintaining their own threat intelligence infrastructure.

5.3 Environmental Impact

SecSuite's serverless edge deployment means compute resources activate only when requests arrive. Idle periods consume no server resources. The two-tier cache (KV at 1-hour TTL, D1 at 24-hour TTL) prevents repeat ML inference on the same URL or article. When a malicious URL has already been analyzed, the cached result returns immediately with no new compute cost.

The analysis pipeline does carry real compute costs. The Malicious Link Evaluation module runs a headless Chromium instance for screenshot capture on each URL scan. The phishing detection stage runs Faster R-CNN, a Siamese network, and ResNet sequentially. The LongFormer script classifier processes up to 4096 tokens per file. These operations are GPU-intensive.

SecSuite reduces wasteful digital traffic indirectly. Bot detection removes automated spam accounts that generate high-volume, low-value requests. Phishing URL blocking stops users from visiting pages that would otherwise consume DNS, CDN, and server resources. Spam email filtering reduces unnecessary delivery infrastructure load.

5.4 Societal Impact

The anonymous user system lets anyone run threat analysis without creating an account. This removes the access barrier for users who encounter a suspicious link and need a quick check. Authenticated users gain history, search, and export features, but the core detection remains universally accessible.

The Malicious Link Evaluation module produces a detailed output per scan: a risk score, a list of risk factors with their point contributions, per-vendor VirusTotal results, SSL certificate details, domain age, DNS record presence, and phishing classification with the matched brand and match type. Users see the evidence behind every verdict. They are not asked to trust a binary block.

Article analysis detects AI-generated and duplicate content. SimHash fingerprinting across posts identifies near-duplicate narratives spread across multiple accounts. The ContentClassifier evaluates articles for manipulative framing. Together these signals give platforms and researchers a structured signal to act on rather than relying on manual review.

6. Contemporary Issues

6.1 Evolving Threat Landscape

Generative AI lets attackers produce phishing emails and web pages that are grammatically correct, contextually specific, and visually convincing. Heuristic filters trained on spelling errors and awkward phrasing are no longer sufficient.

SecSuite addresses this at multiple layers. The ContentClassifier, a fine-tuned DistilBERT model, detects semantic manipulation patterns rather than surface-level text errors. The PhishingPageDetector analyzes screenshots using Faster R-CNN for logo region detection, a Siamese network for logo similarity matching, and ResNet for brand classification. This catches pages that visually impersonate a brand even when the text is clean and the URL appears plausible.

The Malicious Link Evaluation pipeline applies phishing detection on the resolved final URL after following the full redirect chain. Attackers frequently route URLs through legitimate redirect services to bypass static blocklists. SecSuite's design handles this by checking the destination, not the entry point.

BotRGCN detects coordination at the graph level. A bot account that posts normal-looking content still receives elevated risk signals when it sits inside a cluster of suspicious accounts. The Weighted PageRank module propagates bot probability through follower and following relationships, making evasion through isolated account behavior less effective.

6.2 Privacy and Ethical Considerations

SecSuite analyzes user-submitted content including URLs, email text, usernames, and articles. Some of this content relates to identifiable individuals and qualifies as personal data under regulations like GDPR.

The deduplication layer reduces third-party data exposure. A URL or article already in the cache does not trigger new queries to VirusTotal or WHOIS. Only novel submissions generate outbound intelligence requests.

False positives carry real costs. The scoring model penalizes domain age under 30 days (+10 points) and missing SPF or DMARC records (+3 points each). A new legitimate business with a recently registered domain and incomplete DNS configuration receives an elevated score through no malicious intent. The detailed risk factor output helps users identify and contest incorrect classifications, but the risk of reputational harm to legitimate sites remains.

The BotDetector was trained on Twibot-22, a Twitter-specific dataset. Users on other platforms with different behavioral norms face a higher risk of misclassification. The ContentClassifier's sensitivity depends on its training data distribution. If that data overrepresents certain languages, political contexts, or writing styles, the model flags content from those groups at higher rates than others.

6.3 Misinformation and Information Warfare

State and non-state actors use coordinated inauthentic behavior to push unified narratives across social platforms. The infrastructure for these campaigns consists of bot accounts operating in networks.

BotRGCN detects this infrastructure directly. Creation date clustering surfaces accounts created in a coordinated window. Engagement velocity flags artificially rapid first-engagement after posting. Low unique interactor counts despite high apparent engagement indicate coordination. The Weighted PageRank module propagates suspicion through the social graph, surfacing the broader network around a single flagged account.

Article analysis adds a content-layer signal. SimHash fingerprinting detects near-duplicate text spread across accounts, a hallmark of coordinated narrative amplification. The ContentClassifier evaluates framing and structure for patterns associated with fabricated or manipulative content.

SecSuite's SDK allows media organizations, platform trust-and-safety teams, and independent researchers to incorporate these signals into their own workflows. The webhook system delivers analysis results to downstream systems automatically on completion, supporting real-time enforcement pipelines.

6.4 Trust and Transparency in Automated Systems

Every SecSuite analysis response includes a sources array. Each entry contains the source name, its individual score, the raw data returned, and the query timestamp. For URL analysis, the full output includes per-vendor VirusTotal results, SSL certificate details, WHOIS domain age, DNS record presence, Tranco rank, and phishing classification with a matched brand and match type. Users trace every point of the risk score back to a named source.

For example, the Malicious Link Evaluation module makes its scoring logic explicit. Each risk factor is documented with a fixed point value: Safe Browsing flagged adds 30, VirusTotal malicious or suspicious adds 25, phishing detected adds 20, domain age under 30 days adds 10. The output lists which factors applied to a given scan. This is auditable and explainable in a way that a single neural network score is not.

7. New Tools and Technologies

7.1 RoBERTa

RoBERTa is a transformer-based NLP model designed for contextual understanding of text. It was chosen because both phishing email and fake news detection require semantic analysis beyond simple keyword matching. In SecSuite, RoBERTa was used as the core model in both modules to identify deceptive language, manipulative wording, and credibility-related risk patterns.

7.2 ONNX and INT8 Quantization

ONNX is a cross-platform model format that enables more efficient deployment of trained deep learning models. In SecSuite, the ModelOptimizer converted the phishing and fake news models from PyTorch to ONNX and applied INT8 quantization, reducing model size from approximately 1.5 GB to 480 MB. This optimization improved deployment feasibility and runtime efficiency, although it did not fully remove performance bottlenecks in the fake news pipeline.

7.3 Explainable AI (XAI)

Explainable AI was used to make model predictions more transparent and interpretable. In the phishing module, EmailReasoningEngine highlights the risky words or phrases that trigger phishing alerts, while in the fake news module, ExplanationEngine identifies suspicious wording and clickbait-like patterns. This helped SecSuite provide not only a score, but also a justification for the result.

7.4 Node.js

Node.js is the JavaScript runtime used for backend tooling and edge-compatible execution, its purpose is to run TypeScript/JavaScript services and build pipelines efficiently, and within SecSuite it powers the Cloudflare Workers-side runtime and development scripts.

7.5 TypeScript

TypeScript is the primary typed language for application code, its purpose is to enforce compile-time safety and improve maintainability, and within SecSuite it defines strongly typed contracts across backend APIs, frontend UI, and shared SDK modules.

7.6 Hono.js

Hono.js is the lightweight web framework used for edge APIs, its purpose is to provide fast routing and middleware composition, and within SecSuite it handles request flow in the Cloudflare Workers backend.

7.7 Hono/zod-openapi

@hono/zod-openapi is the validation and OpenAPI generation layer for Hono routes, its purpose is to unify schema validation with API documentation, and within SecSuite it keeps endpoint input/output contracts typed and documented automatically.

7.8 Zod

Zod is a schema validation and type inference library, its purpose is to validate runtime payloads while preserving static type information, and within SecSuite it guards API boundaries for request and response correctness.

7.9 Drizzle ORM

Drizzle ORM is the type-safe database access layer, its purpose is to manage SQL queries and schema definitions safely, and within SecSuite it is used with Cloudflare D1 for persistent relational data operations.

7.10 Better Auth

Better Auth is the authentication framework for multi-method sign-in, its purpose is to reduce custom auth complexity and security risk, and within SecSuite it supports email/password, OAuth, magic links, anonymous sessions, and API key flows.

7.11 Pino

Pino is a high-performance structured logger, its purpose is to provide low-overhead observability data, and within SecSuite it logs backend events in machine-readable form for debugging and monitoring.

7.12 Resend API

Resend API is the transactional email delivery service, its purpose is to reliably send auth and account emails, and within SecSuite it is used for verification, password reset, and magic-link messaging.

7.13 React

React is the component-based frontend framework, its purpose is to build reusable interactive UI, and within SecSuite it powers web and desktop (Tauri UI layer).

7.14 TanStack Router

TanStack Router is the client-side routing framework, its purpose is to organize navigation with file-based route structure and splitting, and within SecSuite it controls route composition in the web application.

7.15 TanStack Query (React Query)

TanStack Query is the server-state management library, its purpose is to handle fetching, caching, and background refresh logic, and within SecSuite it synchronizes UI with backend scan and account data.

7.16 Zustand

Zustand is a lightweight client-state store, its purpose is to manage local UI/application state with minimal boilerplate, and within SecSuite it stores user preferences and view-level session state.

7.17 Recharts

Recharts is a React charting library, its purpose is to visualize analytical data clearly, and within SecSuite it renders threat analysis visuals such as bar, line, and radar views.

7.18 Framer Motion

Framer Motion is an animation system for React, its purpose is to provide smooth UI transitions and interaction feedback, and within SecSuite it animates critical workflow and component state changes.

7.19 Tailwind CSS

Tailwind CSS is a utility-first styling framework, its purpose is rapid and consistent UI styling, and within SecSuite it defines responsive design and component-level layout styling.

7.20 shadcn/ui

shadcn/ui is a composable component toolkit built on Radix primitives, its purpose is to speed up consistent UI assembly, and within SecSuite it provides standardized, accessible interface building blocks.

7.21 Radix UI

Radix UI is a set of unstyled accessibility-first UI primitives, its purpose is robust component behavior and ARIA correctness, and within SecSuite it underpins interactive controls used by shadcn/ui.

7.22 CVA (Class Variance Authority)

CVA is a variant management utility for class-based styling, its purpose is to centralize style permutations cleanly, and within SecSuite it standardizes Tailwind variant logic across reusable components.

7.23 Tauri

Tauri is a framework for desktop apps using web frontend plus native backend, its purpose is to deliver lightweight desktop distribution with strong performance, and within SecSuite it packages the desktop client with a Rust core.

7.24 Rust

Rust is the systems language used in Tauri-native layers, its purpose is safe and performant low-level integration, and within SecSuite it handles desktop-side native/system operations.

7.25 FastAPI

FastAPI is the Python API framework used in the ML analysis service, its purpose is to expose high-performance typed endpoints, and within SecSuite it hosts URL scanning and inference orchestration APIs.

7.26 Python

Python is the primary language for the AI/ML engine, its purpose is rapid model integration and data processing, and within SecSuite it implements detection pipelines and service logic for analysis.

7.27 Docker

Docker is the containerization platform for service packaging, its purpose is reproducible runtime isolation, and within SecSuite it packages the AI analysis engine for deployable environments.

7.28 Cloudflare Workers

Cloudflare Workers is the edge serverless compute platform, its purpose is low-latency global API execution, and within SecSuite it hosts edge backend functions and request handling.

7.29 Cloudflare D1

Cloudflare D1 is an edge SQLite database service, its purpose is durable relational persistence near compute, and within SecSuite it stores structured backend records.

7.30 Cloudflare KV

Cloudflare KV is a distributed key-value store, its purpose is fast read-heavy storage and caching, and within SecSuite it supports cache, throttling counters, and quick lookup data.

7.31 Cloudflare R2

Cloudflare R2 is object storage, its purpose is durable storage of binary artifacts, and within SecSuite it stores screenshots and analysis output files.

7.32 Cloudflare Queues

Cloudflare Queues is the async message queue service, its purpose is decoupled background processing, and within SecSuite it powers non-blocking jobs such as analyses and outbound tasks.

7.33 Turborepo

Turborepo is the monorepo build orchestration system, its purpose is cached task execution and pipeline coordination, and within SecSuite it manages multi-package build/test workflows.

7.34 pnpm

pnpm is the JavaScript package manager for the monorepo, its purpose is fast and space-efficient dependency management, and within SecSuite it installs and links workspace packages deterministically.

7.35 Vite

Vite is the frontend dev/build toolchain, its purpose is fast local HMR and optimized production bundles, and within SecSuite it builds web-facing UI projects.

7.36 tsdown

tsdown is the TypeScript package build tool, its purpose is compiling shared TS libraries for distribution, and within SecSuite it builds internal typed packages used across apps.

7.37 Wrangler

Wrangler is the Cloudflare CLI, its purpose is local development and deployment control for Workers resources, and within SecSuite it is used to deploy and manage edge services.

7.38 i18next

i18next is the internationalization engine, its purpose is managing translations and locale resources, and within SecSuite it powers multilingual text delivery.

7.39 react-i18next

react-i18next provides React bindings for i18next, its purpose is seamless translation usage in components, and within SecSuite it connects locale state to UI rendering.

7.40 @hey-api/openapi-ts

@hey-api/openapi-ts is an OpenAPI-to-TypeScript client generator, its purpose is eliminating manual API typing, and within SecSuite it generates typed API clients from backend specs.

7.41 Orama

Orama is a client-side full-text search engine, its purpose is fast in-app search without external infrastructure, and within SecSuite it powers documentation and feature search experiences.

7.42 Uvicorn

Uvicorn is the ASGI server used to run FastAPI services, its purpose is to serve async Python

web apps efficiently, and within SecSuite it runs the URL scanning API in local and containerized environments.

7.43 Pydantic

Pydantic is the data modeling and validation layer, its purpose is strict typed parsing and serialization, and within SecSuite it defines request/response schemas for scan endpoints.

7.44 pydantic-settings

pydantic-settings is configuration management on top of Pydantic, its purpose is validated environment-driven settings loading, and within SecSuite it manages runtime config from .env and deployment variables.

7.45 HTTPX

HTTPX is an async-capable HTTP client library, its purpose is robust outbound API communication, and within SecSuite it calls external intelligence providers during analysis stages.

7.46 Playwright

Playwright is a headless browser automation framework, its purpose is controlled page

rendering and interaction, and within SecSuite it captures website screenshots during scan pipelines.

7.47 python-dotenv

python-dotenv is an environment variable loader for local development, its purpose is easy config bootstrapping from .env files, and within SecSuite it initializes local API keys and settings.

7.48 python-whois

python-whois is a WHOIS query library, its purpose is retrieving domain registration metadata, and within SecSuite it feeds reputation checks like domain age and registrar details.

7.50 dnspython

dnspython is a DNS resolution toolkit, its purpose is querying DNS records programmatically, and within SecSuite it inspects MX/SPF/DMARC and other DNS reputation signals.

7.51 tldextract

tldextract is a domain parsing utility, its purpose is accurate extraction of subdomain/domain/suffix parts, and within SecSuite it normalizes URLs for analysis and correlation logic.

7.52 cryptography

cryptography is a Python cryptographic primitives package, its purpose is secure certificate and key operations, and within SecSuite it supports TLS/certificate inspection workflows.

7.53 LightGBM

LightGBM is a gradient-boosting ML framework, its purpose is fast tree-based model inference, and within SecSuite it supports binary malware classification paths.

7.54 Transformers

Transformers is the Hugging Face NLP/model framework, its purpose is standardized transformer model inference, and within SecSuite it powers script or content-focused ML detection components.

7.55 PyTorch

PyTorch is the deep learning runtime, its purpose is tensor-based model execution, and within SecSuite it runs CPU-based inference for integrated ML models.

7.56 torchvision

torchvision is the PyTorch vision utility package, its purpose is image transforms and model support for CV tasks, and within SecSuite it assists visual phishing/malicious content pipelines.

7.57 Pillow

Pillow is an image processing library, its purpose is image loading and transformation, and within SecSuite it handles screenshot/image preprocessing for downstream analysis.

7.58 NumPy

NumPy is the foundational numerical array library, its purpose is efficient vectorized computation, and within SecSuite it underpins feature processing and model input preparation.

7.59 pandas

pandas is a tabular data processing library, its purpose is structured data manipulation, and within SecSuite it supports dataset and feature engineering workflows.

7.60 scikit-learn

scikit-learn is a machine learning utilities framework, its purpose is preprocessing and classical ML tooling, and within SecSuite it supports pipeline-level data and model helper operations.

7.61 pefile

pefile is a Portable Executable parsing library, its purpose is static binary metadata extraction, and within SecSuite it derives malware-related PE features for detection.

7.62 signify

signify is a signature verification library, its purpose is validating signed binary trust metadata, and within SecSuite it contributes signer/signature checks in file analysis.

7.63 pytest

pytest is the test framework for Python services, its purpose is automated validation of behavior, and within SecSuite it runs integration and pipeline correctness tests.

7.64 Ruff

Ruff is a fast Python linter, its purpose is enforcing code quality and style rules, and within SecSuite it is part of the standard quality gate in CI and local checks.

7.65 Black

Black is a deterministic Python formatter, its purpose is consistent code style normalization, and within SecSuite it keeps backend codebase formatting uniform.

7.66 mypy

mypy is a static type checker for Python, its purpose is finding type-level defects before runtime, and within SecSuite it validates typed API and service code paths.

7.67 pre-commit

pre-commit is a Git hook orchestration framework, its purpose is running checks before commits, and within SecSuite it executes formatting/lint/type/test gates automatically.

7.68 Google Cloud Build

Google Cloud Build is the managed CI build service, its purpose is container build and deployment step orchestration, and within SecSuite it builds images and runs deployment workflows.

7.69 Google Cloud Run

Google Cloud Run is the managed serverless container platform, its purpose is autoscaled HTTP service hosting, and within SecSuite it runs the deployed scanning API.

7.70 Google Artifact Registry

Google Artifact Registry is the container/image repository, its purpose is secure image storage and versioning, and within SecSuite it stores build artifacts used by Cloud Run deployments.

7.71 Google Cloud Storage (GCS)

Google Cloud Storage is object storage for artifacts and datasets, its purpose is durable file storage at scale, and within SecSuite it stores and retrieves ML model artifacts.

7.72 Google Secret Manager

Google Secret Manager is managed secret storage, its purpose is secure secret lifecycle management, and within SecSuite it provides runtime API keys and sensitive configuration to deployed services.

8. Background and Literature Review

8.1 Bot Detection

8.1.1 First-Generation: Heuristics and Rule-Based Systems

Early social bot detection relied heavily on rigid, deterministic heuristics. Researchers and platform engineers deployed simple threshold-based algorithms to flag anomalous behavior. Systems analyzed raw metadata, setting tripwires for superhuman posting frequencies, massive bursts of identical spam messages, or highly skewed follower-to-following ratios. However, these architectures proved highly brittle. Adversaries quickly adapted by programming "sleep cycles" into their botnets and randomizing text outputs to bypass hardcoded rate limits, rendering purely rule-based systems effectively obsolete.[1]

8.1.2 Second-Generation: Traditional Machine Learning and Feature Engineering

To counter algorithmic evasion, the literature shifted toward supervised machine learning classifiers, primarily utilizing algorithms like Random Forests, Support Vector Machines (SVMs), and CatBoost. This era was defined by exhaustive, manual feature engineering. Researchers extracted hundreds of statistical indicators from user accounts, including account age, timezone anomalies, and basic linguistic markers using Bag-of-Words or TF-IDF (Term Frequency-Inverse Document Frequency) vectors. While highly effective against rudimentary spam bots, these models struggled with feature staleness. As malicious actors learned which features the classifiers weighted most heavily, they actively manipulated those specific metrics—such as purchasing aged accounts or artificially inflating engagement—to perfectly mimic human statistical distributions.[1]

8.1.3 Third-Generation: Deep Learning and Semantic NLP

Recognizing that manual feature engineering could no longer keep pace with adversarial adaptation, researchers transitioned to Deep Learning to enable automated representation learning. The most pivotal shift in this generation was the integration of sophisticated Natural Language Processing (NLP). Moving away from naive keyword matching, architectures began leveraging pre-trained Transformer models, such as BERT, RoBERTa, and DistilBERT. These models generate dense, high-dimensional embeddings that capture deep contextual semantics, syntax, and sentiment, allowing systems to identify subtle manipulative framing rather than just flagging prohibited vocabulary. The primary limitation of this generation, however, was its isolated scope; analyzing a user's text and metadata in a vacuum ignores the macroscopic context of how that user interacts within the broader network.[2]

8.1.4 Fourth-Generation: Graph Neural Networks (GNNs) and Network Topology

The current paradigm recognizes a fundamental truth: while modern bots can easily deploy AI to generate flawless human text and spoof their metadata, it is computationally and logistically difficult for them to fabricate organic, macroscopic network placement. Consequently, the literature pivoted to Graph Neural Networks (GNNs), such as Relational Graph Convolutional Networks (BotRGCN) and Graph Attention Networks (GATs). These models map the social network as a massive topology of nodes (users) and edges (interactions like retweets, mentions, and follows). A dominant theme in recent literature is the battle against "graph heterophily"—the phenomenon where bots intentionally follow and engage with legitimate human users to camouflage their automated nature and disrupt standard message-passing algorithms.[1][3][8]

8.1.5 The Current Frontier: Multi-Modal Fusion and the LLM Arms Race

The absolute cutting edge of the literature focuses on hybrid intelligence and adversarial evasion. State-of-the-art architectures no longer rely on a single vector. They synthesize Large Language Models (LLMs) for high-fidelity semantic extraction with GNNs for structural analysis. This ensures that both the content generated by the bot and its topological connections are evaluated simultaneously in a unified embedding space. Recent studies heavily detail the threat of LLM-powered bots utilizing reinforcement learning to actively adjust their language, as well as the identification of critical vulnerabilities in GNNs, such as dynamic topological cloaking attacks.[1][2][3][4][6]

8.1.6 Comparative Analysis and System Contributions

Our system is positioned firmly at the frontier of 4th Generation multi-modal GNN architectures, but it diverges significantly from the current literature in its operational efficiency. While modern frameworks (like BotLGT or advanced RGT variants) typically demand massive, server-grade computing clusters to process billions of parameters and 110GB+ datasets like Twibot-22, our implementation achieves state-of-the-art detection with substantially lower resource overhead. By pairing the robust BotRGCN backbone with highly optimized SQLite database management and dynamic batch processing, we deliver enterprise-grade accuracy on standard consumer hardware.

Furthermore, our research empirically tested two novel architectural extensions designed to push the boundaries of current GNN methodologies:

Weighted Probability Diffusion: We introduced a post-inference layer utilizing a Weighted PageRank algorithm. This allows the model to propagate bot probability scores outward through the user's localized social graph, effectively illuminating hidden adversarial networks and coordinated campaigns that traditional, node-isolated classifiers miss.[7]

Segment-and-Predict (Heterophily Masking): To combat the documented challenge of graph heterophily, we evaluated a "Cluster and Predict" framework to isolate community noise. However, exhaustive empirical testing revealed that this addition did not significantly improve the average accuracy over the pure BotRGCN backbone.

Rather than a failure, this finding is a profound testament to the strength of our preprocessing layer. It indicates that our multi-modal feature fusion—synthesizing RoBERTa embeddings with normalized statistical metadata—is so exceptionally robust that the initial node embeddings inherently overpower topological noise. This discovery allowed us to streamline the final production model, strictly utilizing the original BotRGCN architecture. By shedding unnecessary algorithmic complexity, the system guarantees the minimal runtimes and reduced memory overhead required to strictly adhere to an 8-second processing SLA without sacrificing detection fidelity.

8.2 Phishing Mail Detection

Phishing email detection has evolved significantly from early rule-based spam filtering systems to modern context-aware transformer models. The earliest practical defenses were largely based on manually engineered heuristics such as suspicious sender addresses, urgent keywords, malformed headers, domain blacklists, and embedded-link rules. Public corpora such as the SpamAssassin public mail corpus provided a widely used benchmark for testing spam and phishing-oriented filters, while the Enron Email Dataset became a major source of legitimate corporate e-mails for training and evaluation. The Enron corpus contains about half a million messages from roughly 150 users, making it especially useful as a real-world “ham” baseline in email-security research. [9], [10]

As phishing attacks became more linguistically sophisticated, traditional keyword- and header-based approaches began to show clear limitations. Classical machine learning methods such as Naive Bayes, Support Vector Machines, and Random Forests improved over purely rule-based filters by learning statistical patterns from text and metadata, but they still depended heavily on surface features and often struggled when attackers changed wording while preserving deceptive intent. More recent research has therefore shifted toward BERT-family and transformer-based models, which are better suited for capturing semantic manipulation, impersonation cues, social-engineering language, and contextual urgency. Studies applying BERT-based models to phishing e-mails report that contextual transformers are effective for email phishing classification, and later work has extended this direction toward explainable transformer-based phishing detection. [11], [12], [13]

Another important research trend is the move from single-signal detection toward multi-signal phishing analysis. Modern phishing emails are rarely identified by text alone; instead, robust systems often combine body-text semantics with sender verification signals, link analysis, domain reputation, and authentication checks such as SPF, DKIM, and DMARC. This is important because a realistic phishing campaign may use polished language while still exposing structural inconsistencies in the sender domain or embedded URLs. Recent research also emphasizes explainability, since operational deployment in cybersecurity requires not just a score, but evidence that analysts and users can inspect. Transformer-based models paired with XAI methods have therefore become increasingly attractive for phishing-email pipelines. [12], [13]

SecSuite’s Phishing Email Analysis module is aligned with this modern literature. Rather than relying solely on static heuristics, the module combines email-structure checks with RoBERTa-based semantic analysis and an explanation layer that highlights risky

indicators. This design reflects the direction seen in recent phishing-detection research: treating phishing as a hybrid problem involving language, trust signals, and malicious-link context. From an engineering standpoint, this also makes the module more suitable for real-world usage, because it can return an interpretable verdict instead of a raw binary decision. The result is a system that is better aligned with contemporary phishing threats, especially those generated with fluent language and sophisticated impersonation tactics. [11], [12], [13]

8.3 Fake News Detection

Automated fake news detection emerged as a major research area because false or misleading information spreads rapidly online and can influence political behavior, public trust, and social stability. Early work in this field largely framed the problem as a text-classification task based on linguistic signals such as exaggeration, sentiment intensity, hedging, subjectivity, and rhetorical manipulation. However, progress was initially constrained by the lack of publicly available benchmark datasets. A major turning point came with the release of LIAR, introduced by Wang as a dataset of approximately 12.8K manually labeled short statements collected over a decade from PolitiFact. LIAR became one of the earliest widely used benchmarks for fake-news and claim-verification research. [14]

As the field matured, researchers recognized that misinformation detection cannot be solved through text alone. Fake news often depends not only on the wording of an article, but also on how it spreads, who shares it, and whether external evidence supports or refutes the claims. This broader view motivated the development of FakeNewsNet, a repository that combines news content, social context, and spatiotemporal information. FakeNewsNet helped move the field beyond isolated text classification by explicitly modeling interactions between content and propagation behavior. In parallel, the FEVER dataset expanded the neighboring area of automated fact verification by introducing 185,445 claims labeled as Supported, Refuted, or NotEnoughInfo against evidence from Wikipedia, thereby making evidence-aware verification a more central part of misinformation research. [15], [16]

Dataset scale and diversity also became important issues. Smaller datasets often caused overfitting to narrow topics or publisher-specific artifacts, which made it difficult to generalize to real-world misinformation. To address this, larger article-level corpora such as WELFake were introduced. The WELFake dataset contains 72,134 news articles and was explicitly designed to provide broader text coverage by merging multiple existing sources. This line of work reflects a major lesson in fake-news research: a detector that performs well on one narrow corpus may fail badly when confronted with a new source, style, or topic. As a result, dataset composition and leakage control became just as important as model selection. [17]

Model development has followed the same trajectory. Early systems commonly used TF-IDF features with Logistic Regression, SVMs, or Random Forests. Later work adopted CNNs and RNNs to better model local and sequential textual structure. More recently, transformer-based models such as BERT and RoBERTa have become central because they provide richer contextual representations and are more effective at capturing subtle manipulative framing, semantic inconsistency, and discourse-level signals. At the same time, the literature increasingly argues that classification alone is insufficient. Fact-verification-aware approaches now aim to connect article classification with evidence

retrieval and claim verification, rather than treating fake-news detection as a closed-box pattern-recognition task. [18], [19], [20]

SecSuite’s Fake News Detection module is consistent with this more modern, credibility-oriented view. The module uses a RoBERTa-based detector as its core classifier, but it is conceptually closer to current evidence-aware pipelines than to older binary classifiers. Its overall design emphasizes article credibility, suspicious wording, and explanation, rather than claiming to perform full human fact-checking. This is an important distinction also reflected in the literature: many practical systems are best understood as credibility-assessment and misinformation-risk tools, not as definitive arbiters of truth. By combining article-level analysis with source-aware and explainable outputs, SecSuite follows the same research direction that connects fake-news detection with evidence, context, and transparency. [18], [20]

8.4 Malicious URL Detection

Early approaches to malicious URL detection relied on blacklists maintained by threat intelligence services such as Google Safe Browsing and PhishTank. These systems flag known malicious URLs by matching submitted addresses against continuously updated databases. While effective for known threats, blacklist-based methods fail against zero-day URLs and rapidly rotating domains used in modern phishing and malware campaigns [21], [22].

Lexical analysis emerged as a complement to blacklisting. Researchers observed that malicious URLs share structural patterns: excessive subdomains, long path lengths, the presence of IP addresses in place of hostnames, and the use of URL shorteners. Ma et al. demonstrated that features extracted directly from the URL string, without fetching the page, achieve strong classification accuracy using logistic regression and support vector machines [23]. Subsequent work by Garera et al. applied similar feature engineering specifically to phishing URLs, identifying tokens commonly embedded in fraudulent addresses [24].

Host-based and content-based features expanded the feature space further. Domain registration age, WHOIS data, DNS record configuration, and Autonomous System Number (ASN) information all carry predictive signal. Sahoo et al. published a comprehensive survey covering feature categories across URL, host, and page content levels, and benchmarked multiple classifiers including random forests and gradient boosting on these features [25].

Deep learning shifted the field toward end-to-end models that reduce manual feature engineering. Le et al. proposed URLNet, a convolutional neural network that processes both character-level and word-level URL representations simultaneously, achieving competitive performance without hand-crafted features [26]. Recurrent architectures, particularly LSTMs, were applied by Bahnsen et al. to model the sequential character structure of URLs, with results showing strong generalization across phishing and malware categories [27].

Graph-based and behavioral approaches address evasion techniques that defeat static analysis. Antonakakis et al. introduced Notos, a system that builds reputation scores for domains by analyzing DNS query behavior and passive DNS data [28]. Bilge et al. extended this direction with EXPOSURE, which uses 15 behavioral features derived from DNS traffic patterns to identify malicious domains before they appear on any blacklist [29].

Transformer-based models represent the current frontier. Jiang et al. applied BERT to URL classification, treating the URL as a token sequence and fine-tuning on labeled phishing datasets, reporting improvements over CNN and LSTM baselines [30]. Integrated systems such as the one described in this project combine threat intelligence APIs, certificate analysis, lexical checks, and ML classifiers into a unified pipeline, reflecting the consensus that no single method covers the full threat surface [31].

Commonly used datasets and benchmarks in this field include the ISCX-URL-2016 dataset [32], the PhishTank URL corpus [33], and the Alexa/Tranco top-site lists used as benign baselines. VirusTotal aggregates results from over 70 antivirus engines and serves as a de facto multi-source ground truth for URL and file reputation [34].

9. Test Results and Evaluation

9.1 Test Results

Test ID	Description	Input(s)	Expected Output	Status
TC-01	Valid username — human account	Username: @authentic_user (real, active account)	Bot Likelihood < 30%; classification: Human; network graph rendered	Pass
TC-02	Known bot account detection	Username: @known_bot_account (test bot seed)	Bot Likelihood ≥ 70%; c	Pass
TC-03	Non-existent username	Username: @xyznonexistent123abc	Error message: 'User not found'; no score displayed	Pass
TC-04	Response time within SLA	Any valid username submitted	Response returned within 10 seconds (SLA requirement)	Pass
TC-05	Bot Detection — KV cache hit	Same username submitted twice within 24 h	Second request returns immediately from cache; no new API call	Pass
TC-06	Known phishing email body	Email body containing urgent wire transfer request with spoofed domain	Safety Score ≥ 70; high-risk elements highlighted; classification: Phishing	Pass
TC-07	Legitimate email body	Standard corporate newsletter from a known domain	Safety Score < 30; classification: Safe; no high-risk indicators	Pass
TC-08	Empty email body submitted	Headers provided; body left empty	Validation error: 'Email body is required'	Pass

Test ID	Description	Input(s)	Expected Output	Status
TC-09	Email with embedded malicious link	Body containing a link matching a threat-DB entry	Link flagged as malicious; score elevated; link highlighted in results	Pass
TC-10	Phishing detection response time	Any well-formed phishing email input	Response returned within 8 seconds (SLA requirement)	Fail (avg. 30s)
TC-11	Verified credible article URL	URL of a Reuters or BBC news article	Credibility Score $\geq$ 70; classification: Credible; low bias indicators	Pass
TC-12	Known fake news article text	Raw text from a flagged misinformation source	Credibility Score $<$ 40; classification: Unreliable; linguistic anomalies flagged	Pass
TC-13	Inaccessible URL submission	URL returning HTTP 404	Error message: 'Unable to retrieve article'; graceful failure shown	Pass
TC-14	Raw text input without URL	Pasted paragraph of fabricated news text	Text extracted and assessed; credibility score returned within 8 seconds	Pass
TC-15	Empty text field	No input submitted	Validation error: 'Please enter text or a URL'	Pass
TC-16	Known malicious URL	URL flagged in Google Safe Browsing DB	Status: Malicious (red banner); threat category displayed; screenshot blurred	Pass
TC-17	Safe URL check	https://www.google.com	Status: Safe (green banner); WHOIS age shown; SSL valid; screenshot rendered	Pass
TC-18	URL with redirect chain	Shortened URL redirecting multiple times	Full redirect chain displayed; final destination evaluated	Pass
TC-19	Malformed URL input	Input: 'not-a-url'	Validation error: 'Invalid URL format'	Pass

Test ID	Description	Input(s)	Expected Output	Status
TC-20	Link Checker response time	Any valid URL	Response returned within 8 seconds (SLA requirement)	Fail (avg. 9s)
TC-21	User registration — valid data	New email + strong password	Account created; verification email sent; redirect to dashboard	Pass
TC-22	User registration — duplicate email	Email already registered	Error: 'Email already in use'; no duplicate account created	Pass
TC-23	Login — valid credentials	Registered email + correct password	Session created; JWT issued; user redirected to dashboard	Pass
TC-24	Login — invalid credentials	Registered email + wrong password	Error: 'Invalid credentials'; no session created	Pass
TC-25	Social OAuth login (Google)	OAuth flow initiated; valid Google account	User authenticated; session created; profile populated	Pass
TC-26	Anonymous guest session creation	Guest clicks 'Continue as guest'	Temporary session created; analysis history begins accumulating	Pass
TC-27	Anonymous → Registered transition	Guest with history registers a permanent account	All analysis history transferred; anonymous account deleted; zero data loss	Pass
TC-28	API key creation	Authenticated user creates a new API key	Key returned once (plaintext); hash stored; key visible in dashboard	Pass
TC-29	Rate limit — anonymous user exceeded	Anonymous user sends > 100 requests/day	HTTP 429 returned; Retry-After header included	Pass
TC-30	Rate limit — authenticated user	Authenticated user sends > 500 requests/day	HTTP 429 returned after quota exhausted	Pass
TC-31	CAPTCHA validation on registration	Registration attempt from automated	Request rejected; HTTP 400 returned	Pass

Test ID	Description	Input(s)	Expected Output	Status
		script without CAPTCHA token		
TC-32	SQL injection in input field	Input: ' OR 1=1; --	Input sanitized; no database error; safe error message returned	Pass
TC-33	XSS attempt in text fields	Input: <script>alert('xss')</script>	Input escaped; no script execution; content displayed safely	Pass
TC-34	TLS enforcement	HTTP request to API endpoint	Automatic redirect to HTTPS; all data in transit encrypted	Pass
TC-35	KV hot cache deduplication	Same URL submitted twice within 1 hour	Second response served from KV cache; latency < 200 ms	Pass
TC-36	DB cache fallback on KV miss	Analysis older than 1 h but within 24 h resubmitted	Result retrieved from DB; KV back-populated; correct result returned	Pass
TC-37	Concurrent user load test	100 simultaneous users submitting link-check requests	All requests completed successfully; no 5xx errors; SLA maintained	Pass
TC-38	System uptime monitoring	Continuous uptime check over 24-hour period	Uptime $\geq$ 99.5% (SLA requirement)	Pass
TC-39	History persistence for auth users	Authenticated user submits 5 analyses then navigates to History	All 5 analyses appear in history with correct metadata	Pass
TC-40	Visibility — private analysis	Analysis set to Private by owner	Only owner can view; not returned in public search	Pass
TC-41	Visibility — public analysis	Analysis set to Public	Any user can view via direct link and public search	Pass
TC-42	SSE real-time progress stream	User submits analysis and	Progress events emitted for each phase: queued,	Pass

Test ID	Description	Input(s)	Expected Output	Status
		monitors progress stream	processing, completed	

9.2 Test Assessment

Overall, the testing results indicate that SecSuite achieved a high level of functional correctness and system stability across most of its core and supporting features. Out of the **42** test cases defined in the test plan, **40 passed** and **2 failed**, corresponding to an overall pass rate of approximately **95.2%**. The successful results show that the platform correctly handled bot detection, phishing content classification, fake news credibility assessment, malicious URL evaluation, authentication workflows, rate limiting, security controls, caching, history persistence, visibility rules, and SSE-based progress streaming. The only failed results were both related to **response-time performance requirements** under the non-bot module SLA.

From a module-specific perspective, the **Phishing Email Analysis** module passed all correctness-oriented tests, including phishing classification, legitimate email detection, empty-body validation, and malicious embedded-link handling. However, **TC-10 (Phishing detection response time)** failed because the evaluated implementation returned results in an average of **30 seconds**, exceeding the **8-second** SLA requirement defined for non-bot modules. Similarly, the **Malicious Link Evaluation** module passed its functional tests for malicious URL detection, safe URL evaluation, redirect-chain handling, and malformed-input validation, but **TC-20 (Link Checker response time)** failed because the average response time was **9 seconds**, again above the required **8-second** threshold.

9.2.1 Discussion of Failed Tests and Bugs

Two failed tests were recorded, and both were caused by **performance bottlenecks rather than incorrect functional behavior**. The first failure was **TC-10**, which evaluates whether the Phishing Email Analysis module returns a result within 8 seconds. Although the module successfully classified the submitted phishing email and returned the expected safety assessment, its average response time was approximately **30 seconds**. This indicates that the semantic-analysis pipeline, including text normalization, link extraction, and RoBERTa-based inference, remains computationally expensive under the evaluated configuration. The problem therefore lies not in the correctness of the phishing verdict, but in the runtime cost of the inference workflow.

The second failure was **TC-20**, which evaluates whether the Malicious Link Evaluation module returns a result within the same 8-second limit. Functionally, the module still behaved correctly: it processed valid URLs, followed redirect chains, checked threat-intelligence sources, and produced the expected verdict structure. However, its measured average runtime was approximately **9 seconds**, which was enough to fail the SLA test. Given the design of the module, this delay is understandable because the pipeline combines several expensive steps, including redirect resolution, WHOIS and DNS checks, SSL inspection, screenshot capture through Playwright, VirusTotal/Google Safe Browsing integration, and sequential phishing detection on the captured page image.

In both cases, the failures were **not fully resolved** in the current project version. The system remains functionally usable, and the failed tests do not invalidate the core outputs of either module. However, they show that the platform does not yet meet full SLA compliance for

every non-bot analysis path under the tested environment. This distinction is important: the platform is largely correct in terms of threat-detection logic, but still requires additional performance optimization before it can be considered fully compliant with all test-plan performance targets.

9.2.2 Potential Enhancements

The most important enhancement area is **performance optimization for the two failed response-time tests**. For the phishing module, future work should focus on reducing the cost of preprocessing and inference, especially the combined overhead of HTML cleanup, recursive link extraction, and transformer-based semantic analysis. For the malicious link module, optimization should target the slowest external and sequential stages, particularly screenshot capture, threat-intelligence polling, SSL inspection, and redirect-aware analysis. Profiling each stage separately would make it easier to identify which sub-operations contribute most to total latency.

A second enhancement area is **infrastructure-aware performance testing**. The test plan already notes that GPU availability, shared cloud resources, AI-engine load, and third-party service latency can affect representative timing results. Therefore, future benchmark runs should be repeated under controlled load conditions and preferably during off-peak windows, so that the measured response-time performance can be separated more clearly from temporary infrastructure variability.

Finally, the system would benefit from broader improvements in **model efficiency and deployment strategy**. Additional model compression, lighter explainability strategies, more aggressive caching of repeated results, and better separation of synchronous versus asynchronous tasks could improve SLA compliance without weakening detection quality. These changes would help SecSuite preserve its explainable and multi-stage analysis design while moving closer to the low-latency targets defined in the test plan.

10. Conclusion

10.1 Summary of Achievements

SecSuite was fully implemented across all four client surfaces and all six detection modules. The Social Media Bot Detection module was trained and evaluated on Twibot-22 using the BotRGCN architecture with multi-modal feature fusion combining account metadata, semantic text embeddings, and social graph topology. The Phishing Email Analysis module fine-tuned RoBERTa on a dataset of approximately 82,500 emails and passed all functional test cases. The Fake News Detection module trained RoBERTa on the WELFake dataset, integrated SHAP-based explainability, and applied temperature calibration to address softmax saturation from INT8 quantization. The Malicious Link Evaluation module combined six parallel threat-intelligence checks with a sequential phishing detection pipeline and produced scored, detailed outputs across all tested functional scenarios. Script analysis within the URL scanner successfully classified PowerShell-based malicious scripts using a LongFormer model trained on the MPSD dataset.

The backend deployed to Cloudflare Workers with a two-tier cache, SSE-based progress streaming, API key authentication, and rate limiting. All four client applications, the SDK, and the CI/CD pipeline were completed and operational. Of 42 test cases, 40 passed. Both failures were response-time SLA violations rather than correctness failures, confirming that

the platform meets its original objectives of consolidating multi-vector threat detection into a single normalized API accessible from four client surfaces.

10.2 Lessons Learned

The most significant technical lesson was that ML inference latency must be treated as a first-class design constraint from the start. Both SLA failures originated from the same root cause: deep learning pipelines that are accurate but slow under CPU-constrained edge deployment. Addressing model size, quantization strategy, and batching design early would have allowed earlier detection of these bottlenecks.

A second lesson was the value of strict interface contracts between components. Defining the unified request and response schema for the AI engine early allowed independent development of all four clients and the backend without integration conflicts. Shared TypeScript types enforced these contracts across the monorepo.

Dataset selection directly bounds detection quality. BotRGCN performed well on Twibot-22 because the training data matched the deployment target. The LogoSENSE logo library limits phishing detection to a fixed set of brands, and the MPSD dataset limits script detection to PowerShell. These are dataset constraints, not model failures. Planning for coverage gaps during dataset selection is as important as model selection itself.

All deployed models produce false positives and false negatives at some rate. This was observed across modules during testing and should be factored into any deployment context that depends on automated verdicts. Outputs are risk signals, not ground truth.

10.3 Future Work

The highest-priority direction is performance optimization for the two modules that failed SLA testing. The phishing email pipeline needs stage-level profiling to isolate the costliest preprocessing and inference steps, with candidate solutions including model distillation, caching of intermediate representations, and asynchronous link extraction. The malicious link pipeline needs per-stage benchmarking to identify whether the bottleneck is screenshot capture, VirusTotal polling, or the sequential phishing detection steps, followed by targeted optimization.

A second direction is expanding the phishing detection logo library. The current reference set covers only the brands included in the LogoSENSE dataset. Adding logos for a broader range of targeted brands would directly improve detection coverage without requiring architectural changes.

A third direction is extending script analysis beyond PowerShell. The current classifier covers only PowerShell scripts. Supporting additional types such as Bash, VBScript, and JavaScript would require new labeled training data and separate model pipelines for each script type.

A fourth direction is expanding bot detection coverage beyond Twitter. The current BotRGCN model is trained on Twibot-22, which is Twitter-specific. Supporting other platforms requires platform-appropriate training data and adjustments to the metadata extraction layer.

A fifth direction is real-time integration. The current platform analyzes submitted indicators on demand. Adding email gateway integration, live URL scanning at the browser level, and

webhook-triggered enforcement pipelines would increase operational value for security teams.

A sixth direction is multilingual support. The ContentClassifier and phishing models were trained primarily on English-language data. Extending coverage requires multilingual training data and fine-tuning on models such as XLM-RoBERTa.

References

- [1] Cresci, S. (2020). "A Decade of Social Bot Detection: Services, Methods, and Challenges." IEEE Communications Magazine.
- [2] Devlin, J., et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." NAACL-HLT.
- [3] Feng, S., et al. (2021). "BotRGCN: Twitter Bot Detection with Relational Graph Convolutional Networks." Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM).
- [4] Feng, S., et al. (2022). "TwiBot-22: Towards Graph-Based Twitter Bot Detection." Proceedings of the Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks.
- [5] Kipf, T. N., & Welling, M. (2017). "Semi-Supervised Classification with Graph Convolutional Networks." International Conference on Learning Representations (ICLR).
- [6] Liu, Y., et al. (2019). "RoBERTa: A Robustly Optimized BERT Pretraining Approach." arXiv preprint arXiv:1907.11692.
- [7] Page, L., et al. (1999). "The PageRank Citation Ranking: Bringing Order to the Web." Stanford InfoLab.
- [8] Schlichtkrull, M., et al. (2018). "Modeling Relational Data with Graph Convolutional Networks." The Semantic Web – ESWC 2018.
- [9] Apache SpamAssassin Project, "SpamAssassin Public Corpus," Apache.
- [10] W. W. Cohen, "Enron Email Dataset," Carnegie Mellon University.
- [11] M. Songailaitė, E. Kankevičiūtė, B. Zhyhun, and J. Mandravickaitė, "BERT-Based Models for Phishing Detection," in Proceedings of the Conference and Labs of the Evaluation Forum Workshops, 2023.
- [12] M. A. Uddin and I. H. Sarker, "An Explainable Transformer-based Model for Phishing Email Detection: A Large Language Model Approach," arXiv preprint arXiv:2402.13871, 2024.
- [13] S. K. Birthriya and S. B. Shanmugam, "A Comprehensive Survey of Phishing Email Detection and Protection Techniques," 2022.
- [14] W. Y. Wang, "Liar, Liar Pants on Fire: A New Benchmark Dataset for Fake News Detection," in Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), 2017.
- [15] K. Shu, D. Mahudeswaran, S. Wang, D. Lee, and H. Liu, "FakeNewsNet: A Data Repository with News Content, Social Context and Spatialtemporal Information for Studying Fake News on Social Media," 2018.

- [16] J. Thorne et al., "FEVER: a Large-scale Dataset for Fact Extraction and VERification," in Proceedings of NAACL-HLT, 2018.
- [17] A. H. Fakey, B. M. Ahmed, and S. A. Yousif, "WELFake: a Large-scale Dataset for Fake News Detection," Zenodo, 2021.
- [18] Q. Li, Q. Hu, and A. Wang, "Connecting the Dots Between Fact Verification and Fake News Detection," in Proceedings of COLING, 2020.
- [19] M. Farokhian and A. R. Gholamian, "Fake News Detection Using Parallel BERT Deep Neural Network," arXiv preprint arXiv:2204.04793, 2022.
- [20] S. F. N. Azizah, H. D. Cahyono, S. W. Sihwi, and W. Widiarto, "Performance Analysis of Transformer Based Models (BERT, ALBERT and RoBERTa) in Fake News Detection," arXiv preprint arXiv:2308.04950, 2023.
- [21] Google LLC, "Google Safe Browsing API," Google Developers, 2024. [Online]. Available: <https://developers.google.com/safe-browsing>
- [22] PhishTank, "PhishTank Developer Information," OpenDNS, 2024. [Online]. Available: https://www.phishtank.com/developer_info.php
- [23] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Paris, France, 2009, pp. 1245-1254. [Online]. Available: <https://dl.acm.org/doi/10.1145/1557019.1557153>
- [24] S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," in Proc. ACM Workshop on Recurring Malcode (WORM), Alexandria, VA, USA, 2007, pp. 1-8. [Online]. Available: <https://dl.acm.org/doi/10.1145/1314389.1314391>
- [25] D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," arXiv preprint arXiv:1701.07179, 2017. [Online]. Available: <https://arxiv.org/abs/1701.07179>
- [26] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," arXiv preprint arXiv:1802.03162, 2018. [Online]. Available: <https://arxiv.org/abs/1802.03162>
- [27] A. C. Bahnsen, E. C. Bohorquez, S. Villegas, J. Vargas, and F. A. Gonzalez, "Classifying phishing URLs using recurrent neural networks," in Proc. APWG Symp. Electronic Crime Research (eCrime), Scottsdale, AZ, USA, 2017, pp. 1-8. [Online]. Available: <https://ieeexplore.ieee.org/document/8110035>
- [28] M. Antonakakis, R. Perdisci, D. Dagon, W. Lee, and N. Feamster, "Building a dynamic reputation system for DNS," in Proc. 19th USENIX Security Symp., Washington, DC, USA, 2010, pp. 273-290. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity10/building-dynamic-reputation-system-dns>
- [29] L. Bilge, E. Kirda, C. Kruegel, and M. Balduzzi, "EXPOSURE: Finding malicious domains using passive DNS analysis," in Proc. 18th Annual Network and Distributed System Security Symp. (NDSS), San Diego, CA, USA, 2011. [Online]. Available: <https://www.ndss-symposium.org/ndss2011/exposure-finding-malicious-domains-using-passive-dns-analysis>

- [30] N. Jiang, G. Yu, and F. Tao, "Malicious URL detection based on pre-trained BERT model," in Proc. 2020 IEEE 5th Int. Conf. Data Science in Cyberspace (DSC), Hong Kong, China, 2020, pp. 127-132. [Online]. Available: <https://ieeexplore.ieee.org/document/9172880>
- [31] VirusTotal, "VirusTotal API v3 Overview," Google LLC, 2024. [Online]. Available: <https://developers.virustotal.com/reference/overview>
- [32] M. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, "Detecting malicious URLs using lexical analysis," in Proc. Int. Conf. Network and System Security (NSS), Taipei, Taiwan, 2016, pp. 467-482. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-46298-1_30
- [33] PhishTank, "PhishTank Statistics and Data," OpenDNS, 2024. [Online]. Available: <https://www.phishtank.com/stats/>
- [34] H. Shirazi, B. Bezawada, and I. Ray, "Know thy domain name: Unbiased phishing detection using domain name based features," in Proc. 23rd ACM Symp. Access Control Models and Technologies (SACMAT), Indianapolis, IN, USA, 2018, pp. 69-75. [Online]. Available: <https://dl.acm.org/doi/10.1145/3205977.3205992>

Appendices

Appendix A: API Documentation

<https://url-scanning-api-601419501974.europe-west1.run.app/docs>
<https://secsuite-fake-news-api-852805197313.europe-west1.run.app/docs>
<https://secsuite-phishing-api-9490606507.europe-west1.run.app/docs>
<https://botrgcn-api-test-14571149912.europe-north1.run.app/docs>
<https://api.ss.archyn.net/reference>
<https://ss.archyn.net/docs>

Appendix B: Dataset Descriptions

Phishing Email Analysis Dataset

The primary dataset used for training and evaluating the **Phishing Email Analysis** module is the **Phishing Email Dataset** published on Kaggle by **Naser Abdullah Alam** and **Amith Khandakar**. The main training file is **phishing_email.csv**, which the dataset page describes as a **combination of six source datasets**. The same page states that the central feature in the final consolidated file is the **text_combined** column, which merges the **subject line, body text, date, and sender information** into a single text field for classification. This structure makes the dataset especially suitable for transformer-based models such as **RoBERTa**, since it preserves broader contextual information instead of limiting the model to body text alone.

Dataset Composition and Statistics:

- **Total Samples:** The dataset contains approximately **82,500 e-mails** in total; Kaggle results and notebooks using the same dataset indicate an effective combined size of **82,486 samples** in phishing_email.csv.
- **Class Distribution:** The dataset contains **42,891 phishing/spam e-mails** and **39,595 legitimate e-mails**, which makes it relatively balanced for supervised binary classification.
- **Features:** In the final consolidated file, the core columns used by many Kaggle implementations are **text_combined** and **label**. The text_combined field represents merged e-mail content, while label provides the binary target for phishing-versus-legitimate classification.
- **Source Composition:** Kaggle notebooks working directly with this dataset show that the package includes six component CSV files: **SpamAssasin.csv**, **Nazario.csv**, **Nigerian_Fraud.csv**, **CEAS_08.csv**, **Enron.csv**, and **Ling.csv**, in addition to the final merged phishing_email.csv file.
- **Auxiliary Fields in Source Files:** The component files preserve richer e-mail structure such as **sender**, **receiver**, **date**, **subject**, **body**, and **URLs**, which shows that the merged dataset is grounded in both textual and metadata-level information.

Fake News Detection Dataset

The primary dataset used for training and evaluating the Fake News Detection module is the **WELFake Dataset**. This dataset is a comprehensive collection designed specifically for disinformation research, merging several smaller datasets to provide a robust and diverse set of linguistic patterns.

Dataset Composition and Statistics:

- **Total Samples:** The dataset contains **72,134** unique news articles.
- **Class Distribution:** The data is well-balanced to ensure the model does not develop a majority-class bias, consisting of **35,028 (48.6%)** real news articles and **37,106 (51.4%)** fake news articles.
- **Features:** Each record includes the article's *title*, full *text* body, and a binary *label* indicating its credibility (1 for fake, 0 for real).

EMBER2024 Dataset

EMBER2024 is a large-scale benchmark dataset for malware classification, covering multiple executable file types across Windows, Android, Linux, and PDF formats.

It is an update to the original EMBER2017 and EMBER2018 datasets, containing raw features and labels for 3.2 million malicious and benign files across six file types: Win32, Win64, .NET, APK, ELF, and PDF. Files were sampled from VirusTotal between September 2023 and December 2024, with the first 52 weeks forming the training set and the last 12 forming the test set. This split lets researchers simulate detection of malware newer than the training corpus. The training set holds 2,626,000 files and the test set holds 606,000. The dataset supports seven label types including malicious/benign classification, malware family, behavior prediction, and packer detection. It also includes a challenge set of 6,315 files that initially evaded roughly 70 antivirus products but were later confirmed malicious, making it useful for testing detection of evasive samples.

MPSD Dataset

MPSD (Malicious PowerShell Script Detection) is a labeled dataset of PowerShell scripts for training classifiers to identify script-based attacks.

PowerShell is a command-line programming language built on the .NET framework. Hackers frequently use malicious PowerShell scripts in APT attacks to control victim machines or install backdoors. The dataset contains three categories of samples: pure malicious scripts, benign scripts, and mixed malicious scripts where a malicious sample is merged with a benign one to obscure its features. This mixing is designed to weaken detection signals at the abstract syntax tree node and token levels, making the scripts more complex to classify. It was published alongside a hybrid-feature detection model that analyzes text characters, functions, tokens, and AST nodes. The model achieved 97.76% accuracy on the mixed dataset using five-fold cross-validation and 98.93% on pure original scripts.

LogoSENSE Dataset

LogoSENSE is a bounding-box annotated logo image dataset built for training and benchmarking computer vision models used in phishing detection.

It supplies a corpus of logos from 15 highly phished brands, providing a ground truth dataset for evaluating logo-based anti-phishing studies. The dataset contains 1,530 training samples (102 phishing page snapshots per brand) and 1,979 testing samples (979 brand-logo images plus 1,000 legitimate pages). An additional 102 distractor samples per brand bring the total training size to 3,060 samples. Annotations are provided in two formats, dLib and PASCAL VOC 2007, allowing compatibility with frameworks including Caffè, PyTorch, TensorFlow, and Keras. The dataset was developed at Hacettepe University's HUMIR Lab and is intended strictly for academic use. The state-of-the-art baseline using a HOG-based logo detection scheme achieved a precision of 93.50%, recall of 77.94%, and F1 score of 85.01.

Appendix C: Individual Contributions

Team Member	Contributions
Umut Bayram	Developed Malicious Link Evaluation Model
Kıvanç Taş	Developed Social Bot Detection Models and Methodologies
Archyn Mikhailov	Developed All Frontends, All Backends, CI/CD, UI Components, Uptime, SDK,
Yiğit Sarı	Developed Fake News and Phishing Mail Detection Model